

Manejador de Recursos Distribuidos para HPC.

**Franco Franzini,
Santos David Lurati,
Gianfranco Morán,
Maximiliano Zambón.**

Sistemas Operativos.

**Facultad de Ciencias Exactas
Ingeniería y Agrimensura.**

22/6/2026.

Introducción:

El presente informe documenta el diseño y la implementación del manejador de recursos distribuidos para High-Performance Computing. El cluster HPC es simulado. El objetivo principal del proyecto es construir un nodo autónomo capaz de interoperar con otros nodos en la red, abordando los desafíos que conlleva la sincronización distribuida, el riesgo de deadlocks, y demás.

Para resolver esta complejidad, la arquitectura del nodo se divide en dos componentes especializados que se comunican mediante una interfaz local:

- **Resource Manager Agent (C):** Actúa como el administrador de los recursos físicos del nodo. Utiliza epoll para gestionar de forma concurrente y no bloqueante múltiples conexiones TCP. Además de manejar los recursos locales, es el encargado de interactuar con agentes remotos y de descubrir dinámicamente otros nodos en la red mediante broadcast UDP.
- **Job Scheduler (Erlang):** Funciona como el cerebro planificador. Se conecta al agente C para generar la carga de trabajo simulada, tomar decisiones sobre qué recursos solicitar en el clúster y, fundamentalmente, aplicar la estrategia lógica para prevenir o detectar los deadlocks distribuidos.

A lo largo del informe se detalla cómo se ensamblan ambos módulos utilizando los protocolos definidos, las estructuras de datos empleadas para el manejo de estado y la estrategia adoptada para garantizar que el clúster opere de manera fluida y tolerante a bloqueos.

Roles:

Ingeniero de comunicaciones en C: Maximiliano Zambón.

Gestor de recursos y estado: Santos Lurati.

Planificador y lógica anti-deadlock: Gianfranco Morán/ Franco Franzini.

Integración, pruebas e interoperabilidad: Gianfranco Morán/ Franco Franzini.

Planificador en Erlang:

El programa comienza en el módulo main, en la función main. Primero se crea y registra el logger globalmente, en el cual registramos denegaciones de recursos a trabajos, timeouts, errores del scheduler y demás.

El módulo logger tiene la función de creación del archivo .log, la escritura y cierre del mismo.

Una vez creado el logger se intenta la conexión con el servidor del agente de C, en caso de que no se pueda lograr la conexión el programa lo intentará nuevamente con una cierta espera. Por otro lado si esta se logra establecer, se genera el proceso scheduler, quien espera la creación del oyente del socket de C. Una vez creado este último, el scheduler comienza a trabajar.

El proceso principal, una vez que crea el scheduler, se queda esperando a que el proceso scheduler le envíe una solicitud para cerrar el sistema, y si esta llega la conexión se intenta hacer nuevamente.

Antes de hablar del scheduler, que es el proceso principal, hablemos del oyente del socket en C. El oyente del socket del Agente se encuentra en el módulo tcpClient, este módulo es el que tiene todas las funciones encargadas de la conexión y envío/recibo de mensajes con el agente de C, además tiene un parseo inicial de los mensajes de C.

El proceso scheduler inicia solicitando los nodos que hay en la red, luego crea dos procesos, un generador de trabajos y un actualizador de nodos. El primero enviará solicitudes de creación de trabajo y el segundo solicitará la actualización de los nodos, ambos cada cierto tiempo. Con esto se ejecuta la función del scheduler propiamente dicho. Este permanece constantemente esperando mensajes tanto del agente de C, como las interacciones con el trabajo simulado y la petición de nodos.

En la sección de interacción con nodos, si se encuentra un error al realizar el parseo de los nodos, se utiliza la lista vieja obtenida hasta ese momento. En caso de éxito en el parseo, se calculan los recursos totales del clúster con la función del módulo manejoRecursos correspondiente. El objetivo del cálculo de recursos totales es para que el trabajo simulado no se exceda en la toma de recursos.

Hablemos ahora de las interacciones con el trabajo simulado, si el scheduler obtiene una solicitud para generar trabajo se crea un proceso que actuará como Job, este proceso usa funciones del módulo jobWorker. Este módulo genera cantidades aleatorias de recursos a pedir y espera la respuesta del scheduler con el mensaje del agente, es decir si el Job solicitado dio granted, denied o timeout. Además este módulo anota en el log lo que pasa con ese Job. Por otro lado el job simulado pide al scheduler recursos, quien mediante la función asignarNodos del módulo manejoRecursos lo gestiona, dando una lista de requerimientos concretos para enviar al agente de C. Por último si el Job termina correctamente envía jobTerminado y el scheduler lo recibe. Ahora bien si hay un cierre inesperado del proceso del Job simulado, se lo atrapa para poder limpiar el mapa.

Por último, se tiene la interacción del scheduler con el agente de C. Este espera los mensajes del agente y todo lo que hace es “redirigir” el mensaje al trabajo simulado en caso de que la respuesta sea correcta o si la respuesta es distinta, ya sea por algún comando desconocido o un error en la conexión con el oyente, maneja la situación.

Estrategia elegida contra Deadlock:

Se decidió tomar los recursos como fue convenido en el curso: CPU, Memoria, GPU. En un caso perfecto de que todos los nodos tomen los recursos de la misma manera, esta podría ser la única estrategia contra deadlock implementada, sin

embargo, al no estar seguros de que todos los nodos de los distintos grupos los tomen de la misma manera se implementó un timeout, evitando el bloqueo del sistema a cambio de la posible ocurrencia de livelock.

El ejemplo de deadlock se evita ya que desde el vamos dos Jobs nunca tomarían recursos de la forma CPU -> GPU y otro GPU -> CPU, ambos harían CPU -> GPU. Esto elimina la espera circular. Como hemos mencionado anteriormente este es un caso perfecto, supongamos entonces que el Job1 pide CPU y después GPU, tanto que el Job2 pide GPU y después CPU, en este caso si el Agente A concede CPU a Job1 y el Agente B concede GPU al Job2 entonces si nuestro agente es el A, cuando note que se esta tardando mas del tiempo estipulado en dar la GPU, enviara un timeout al Job1, quien desde erlang, soltará los recursos y intentara volver a ejecutarse después de un cierto tiempo.

Gestor de recursos en C:

La implementación principal del gestor de recursos en C es `gestor_estado.c`, esta representa la capa de abstracción mas alta, la misma se encuentra implementada con los mecanismos de sincronización necesarios para permitir que el `epoll` sea multihilos evitando race conditions. Su integración al mecanismo de comunicación se encuentra en `controlador.c`, este apartado parsea los mensajes recibidos para poder realizar correctamente las respectivas llamadas a función que `gestor_estado` le brinda.

El gestor de estado cuenta con las implementaciones `nodos.c`, `recursos.c`, `transacciones.c` y `jobs.c`. La primera se encarga de llevar registro de los nodos conectados (mediante una tabla hash y una lista simplemente enlazada) para esto implementa la funcionalidad de procesar `announce` añadiendo a la tabla un nodo nuevo o reiniciando el timestamp de uno ya existente, además implementa el `get_nodes`, volcando el correspondiente mensaje de salida en un buffer, por último también implementa el protocolo de desconexión de aquellos nodos que no hayan realizado el `announce` en el tiempo correspondiente, la función correspondiente debe llamarse periódicamente desde el bucle del `epoll` para garantizar la actualización de la tabla.

`recursos.c` implementa la estructura de un recurso local, pudiendo ser `cpu`, `gpu`, `mem` y las solicitudes pendientes de un recurso. Implementa solo la creación y destrucción de su estructura, quedando la lógica de encolación de solicitudes para `jobs.c`. Este último, lleva registro de los jobs mediante una tabla hash y una lista, registra las asignaciones de recursos locales a los jobs, encolando la solicitud del recurso correspondiente si el mismo no está disponible en el momento, y en contrapartida liberando las solicitudes que ya lleven más de 30 segundos sin ser satisfechas, para esto también decidimos chequear las solicitudes encoladas periódicamente y de esa forma evitar un comportamiento lazy sobre las mismas, donde el programa no las denegará, por más que ya haya transcurrido el tiempo, hasta que llegue un receive. También registra las liberaciones de recursos, eliminando el registro del job si es que ya liberó todos los recursos que consumía. Implementa la liberación de los recursos consumidos por jobs de cierto nodo, para de esta forma poder manejar las desconexiones abruptas.

Finalmente transacciones.c implementa las peticiones de múltiples recursos, de esta forma ante una petición rechazada (denied) permite liberar los recursos que ya fueron otorgados localmente o enviar el release de dichos recursos a los nodos correspondientes. Esto es necesario para evitar que los recursos queden reservados en un job que no se puede satisfacer.

Tanto en el registro de los nodos activos como en el de los trabajos activos se almacena en una tablahash y en una lista para mantener la búsqueda de manera eficiente pero permitir su recorrido.

Comunicación en C

La comunicación principal de nuestro agente con otros agentes C remotos, se inicializa y realiza en el archivo agente.c. Este tiene el main, que se encarga de crear e inicializar la estructura EPOLL, los sockets de escucha TCP tanto local como de red, el socket de escucha UDP, los timers necesarios y el estado global del agente con sus recursos disponibles. Luego, se agregan estos FDs al EPOLL para poder monitorear sus eventos, pero no se agregan como FD simplemente, sino que se agregan como una estructura la cual está especificada en cliente.h. Una vez todo iniciado, se crean 4 hilos que se encargaran de correr el bucle principal, donde atenderan los eventos del EPOLL y realizaran la lógica correspondiente dependiendo el evento. Es decir, si llega un cliente que se quiere conectar a los sockets de escucha este se acepta y se agrega su socket de cliente al EPOLL para luego ser atendido, si es el timer que marca el tiempo del announce se arma y envía el mensaje correspondiente, si es un cliente que envía mensajes estos se leen y luego se procesan, etc.

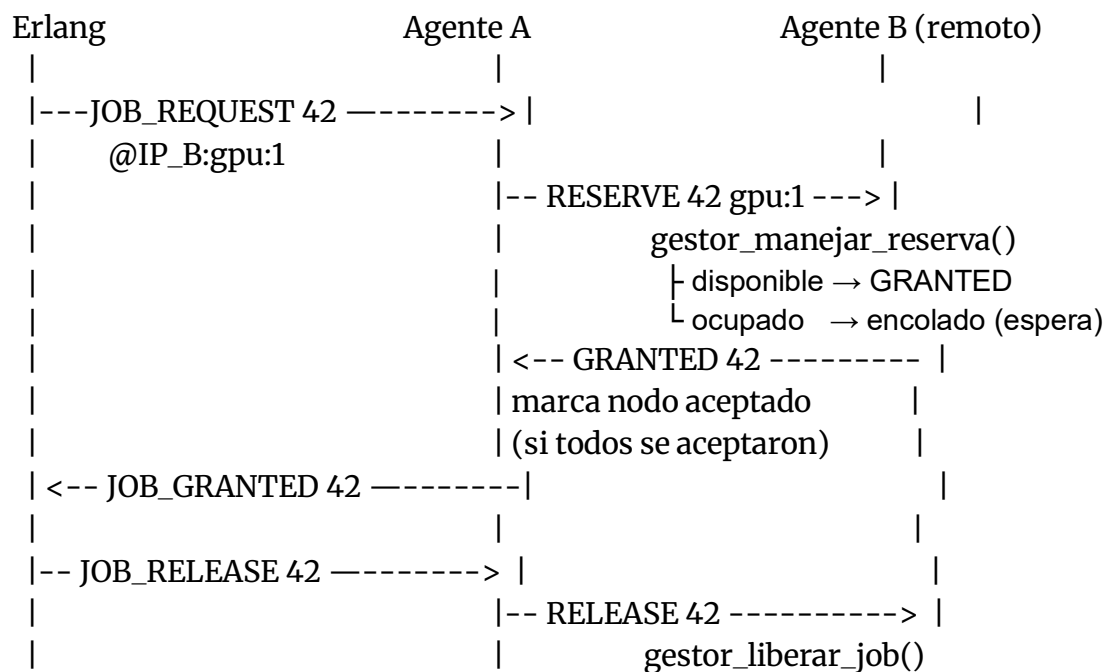
En conjunto con estas, se utilizan las implementaciones sockets.c, cliente.c y controlador.c.

La primera provee la abstracción e implementación de todo lo relacionado a la capa de red. Esta nos ofrece funciones para crear sockets de escucha y timers, conectar el agente a otro nodo por red, enviar y recibir mensajes tanto por TCP como por UDP y saber la id del nodo la cual es necesaria para que este anuncie que está vivo. Todo esto sabiendo solamente a quién y qué mensaje se quiere enviar, aislando al agente de toda la implementación.

Siguiendo con cliente.c, esta solo contiene la estructura ClienteConectado y su función crear. La estructura es utilizada para agregar a los FD que representan a los sockets información necesaria para la recepción de mensajes. Esta guardará 4 datos, el fd que representa al socket de la conexión, un entero que indica si la conexión fue por erlang (local del agente) o por un agente externo y un buffer donde se guardarán los mensajes llegados más un entero que marca el tamaño del mismo. Esta es necesaria pues, en caso de que los mensajes lleguen partidos nos permite ir rearmando el mensaje guardando lo llegado en el buffer del cliente y así asegurarnos a través del epoll que esa información se mantendrá intacta hasta el momento donde se quiera procesar.

Por último, en controlador.c se realiza todo el procesamiento de los mensajes para luego integrar al agente con el gestor de recursos. En este se realiza un parseo del mensaje para distinguir qué petición tanto local como de red llegó y en base a esta poder determinar la decisión a llevar a cabo. Por ejemplo, si llegan requerimientos del erlang, se filtra qué mensaje este nos envió. Si es un job_request, nos quedamos con la información de a qué nodo se está pidiendo que recursos y en qué cantidad para luego poder armar el mensaje reserve a enviar al nodo obtenido. Si es un release, enviamos los release correspondientes a cada nodo que previamente nos había otorgado recursos. Siguiendo con el mismo funcionamiento y lógica para el resto de instrucciones tanto locales como de red

Diagrama de secuencia de la comunicación



Donde el comportamiento del agente A y el agente B es el comportamiento de nuestro agente cuando este actúa como cliente o como proveedor